

Topic-1 JavaFX GUI: Overview of AWT & Swing APIs, and their

Limitations Overview of AWT (Abstract Window Toolkit) API

- **Purpose:** AWT is the original Java GUI toolkit introduced in Java 1.0. It provides a set of GUI components like buttons, text fields, checkboxes, and menus to create basic user interfaces.
- **Platform Dependency:** AWT components are "heavyweight" because they rely on native, platform-specific peer components for rendering. This approach ensures that the components have a look and feel consistent with the operating system.
- **Components:** Basic GUI components such as Button, Label, TextField, Checkbox, and Choice.
- **Layout Managers:** AWT includes layout managers like FlowLayout, BorderLayout, GridLayout, etc., to arrange components in containers.
- **Event Handling:** AWT was the first to introduce the event delegation model to manage user interactions, where events are generated by components and handled by listener interfaces.

Overview of Swing API

- **Purpose:** Swing was introduced in Java 1.2 as part of the Java Foundation Classes (JFC) to offer a more flexible, consistent, and platform-independent GUI toolkit.
- **Lightweight Components:** Unlike AWT, Swing components are "lightweight" because they do not rely on native peers, allowing for more consistent cross-platform behavior.
- **Pluggable Look and Feel:** Swing provides a "pluggable look and feel," enabling applications to use different themes (e.g., Windows, Metal, Motif).
- **Components:** Offers richer and more sophisticated components, like JButton, JLabel, JTextField, JTable, JTree, and JTabbedPane.
- **MVC Architecture:** Swing components follow the Model-View-Controller (MVC) design pattern, where the data model, view, and controller are separated, making customization easier.
- **Event Handling:** Similar to AWT, Swing uses an event delegation model but has more event classes and listener interfaces for finer-grained control.

Limitations of AWT and Swing APIs

While both AWT and Swing have played significant roles in Java GUI development, they come with certain limitations:

- **Performance Issues:** Swing can be slower than native GUI frameworks due to its complex rendering. AWT's reliance on native peers can also cause performance inconsistencies across platforms.
- **Look and Feel Consistency:** AWT is heavily dependent on the underlying operating system's look and feel, leading to inconsistencies. Swing's look and feel, while customizable, may still appear outdated compared to modern UI standards.
- **Lack of Modern UI Components:** Both AWT and Swing lack modern UI components and advanced graphical capabilities, which can limit the creation of sophisticated, visually appealing interfaces.
- **Threading Model:** Swing is single-threaded, meaning all updates to the UI must occur on the Event Dispatch Thread (EDT). If not handled properly, this can lead to thread-related issues and unresponsive applications.

- **Limited Support for Multimedia:** AWT and Swing offer limited support for multimedia components like video and audio, requiring external libraries for complex media handling.
- **Complexity for Developers:** Swing, with its deep component hierarchy and extensive API, can become complex for developers to design, manage, and customize.

Topic-2: Introduction to JavaFX and installation steps

JavaFX was introduced to overcome these limitations, offering a modern, feature-rich GUI toolkit for Java:

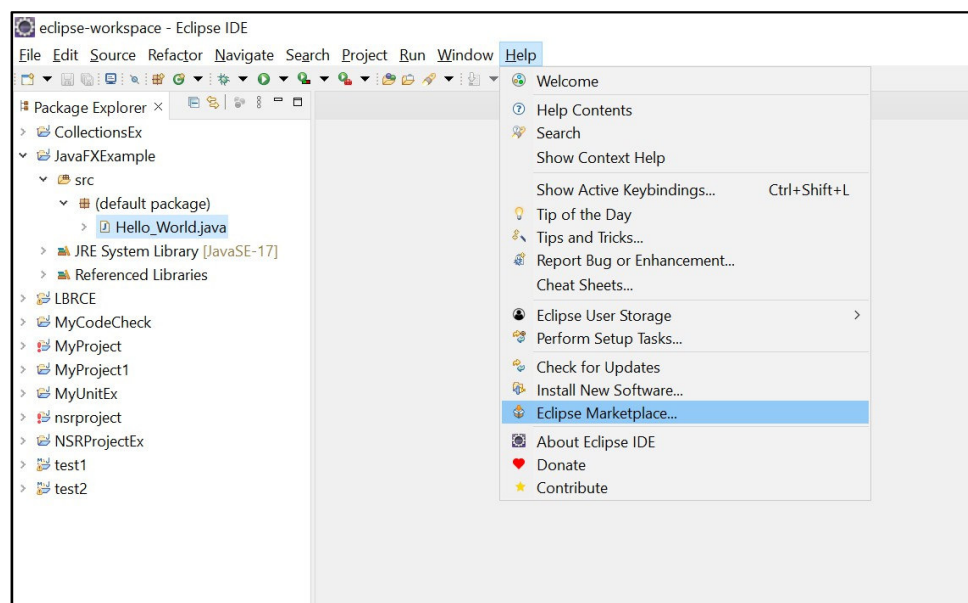
- **Modern UI Capabilities:** JavaFX provides better support for advanced UI components, 2D and 3D graphics, animations, and multimedia.
- **CSS Styling:** Allows for CSS-based styling, enabling developers to create modern-looking applications.
- **FXML:** JavaFX uses FXML, a declarative XML-based language, for building UIs, which helps in separating design and logic, making the codebase more maintainable.
- **Rich Animation and Multimedia Support:** JavaFX has in-built support for animations and multimedia, ideal for creating visually engaging applications.
- **Web Support:** JavaFX includes a WebView component for embedding web content, which is not available in AWT/Swing.

What is JavaFX?

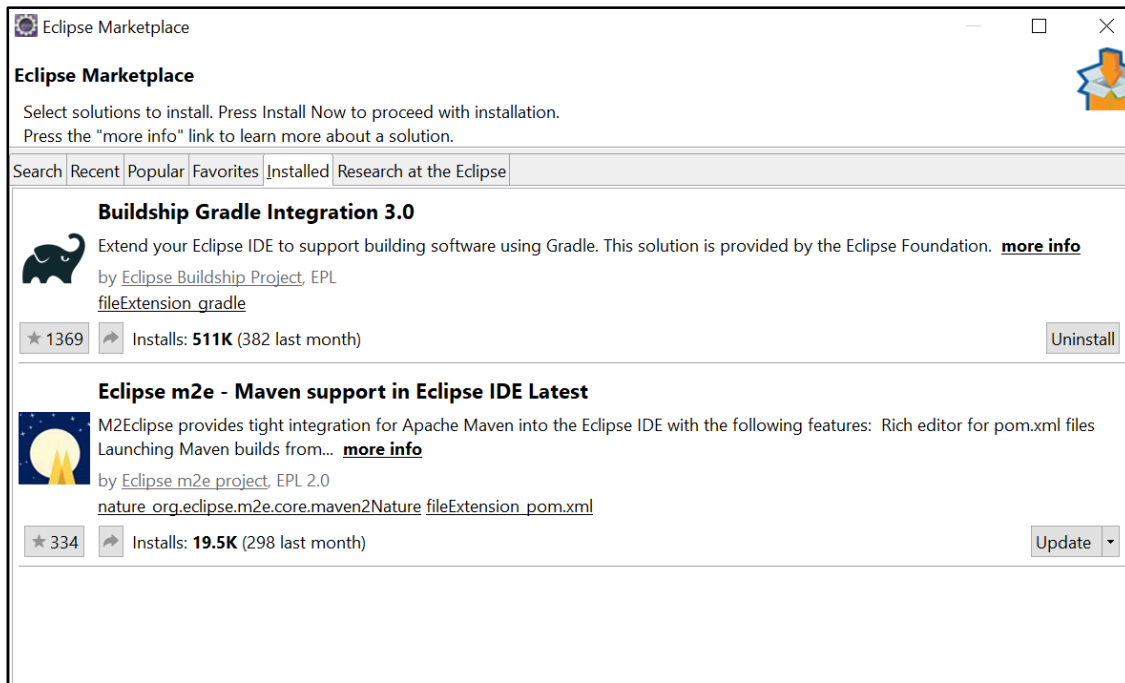
JavaFX is a Java library used to develop Desktop applications as well as Rich Internet Applications (RIA). The applications built in JavaFX, can run on multiple platforms including Web, Mobile and Desktops.

Install JavaFX in Eclipse IDE

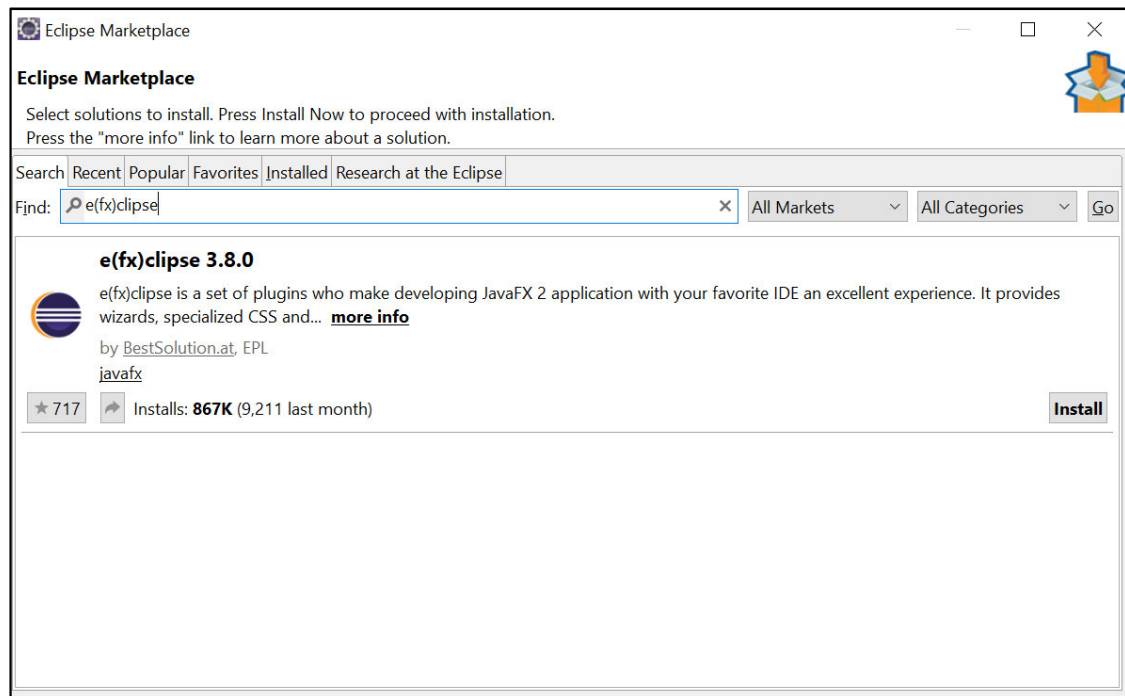
Open Eclipse and click on Help. Choose Eclipse Market place in the list of options given in the drop-down menu.



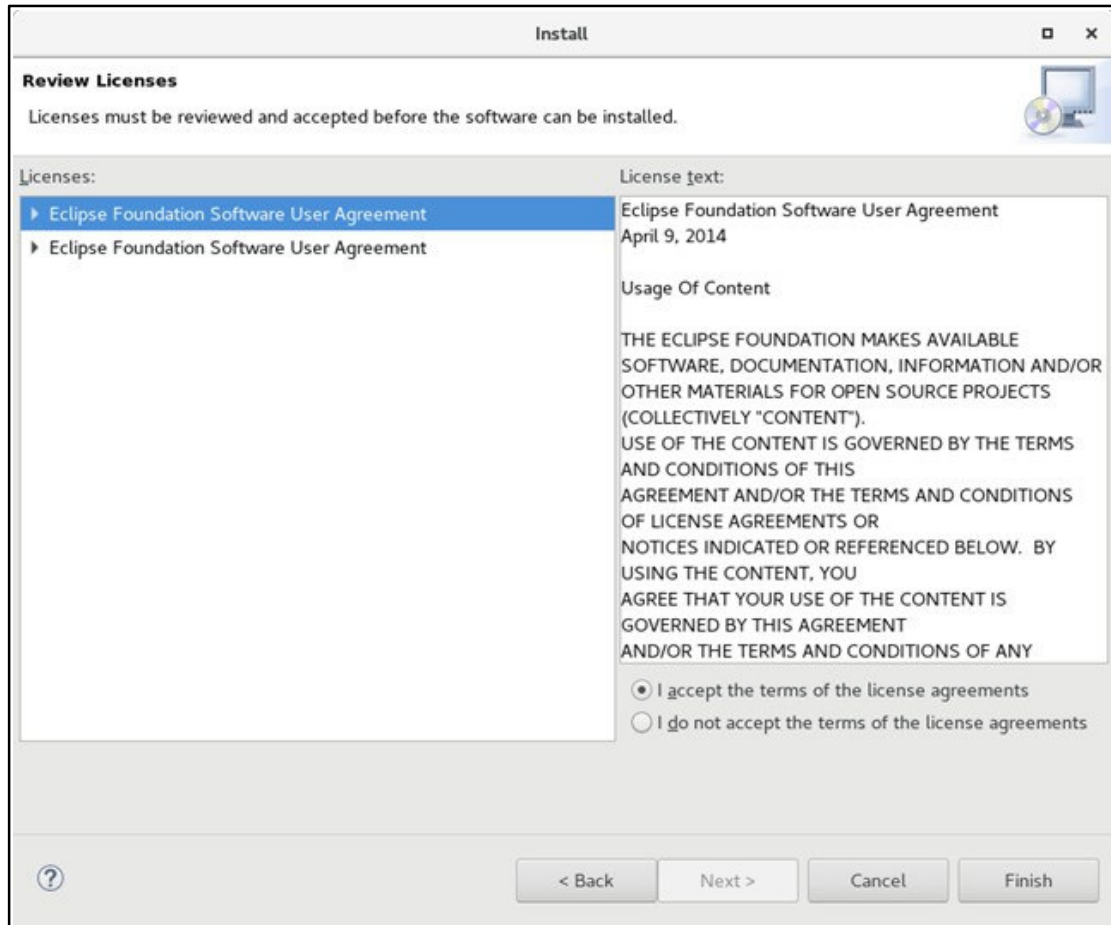
Check whether the e(fx)clipse installed or not



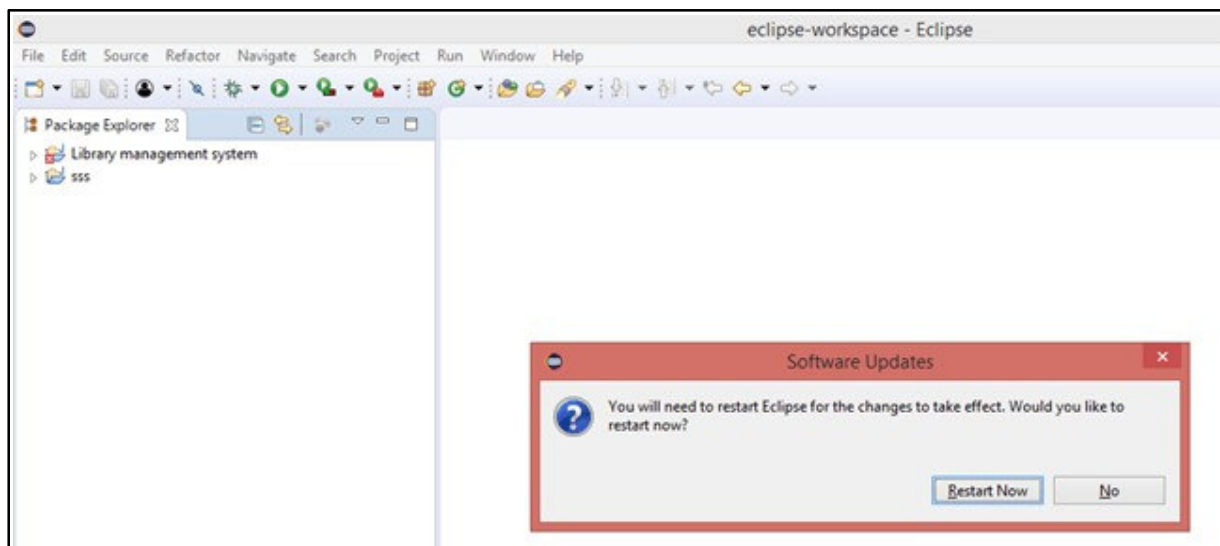
If e(fx)clipse not installed, please go through the search option and type e(fx)clipse and install it.



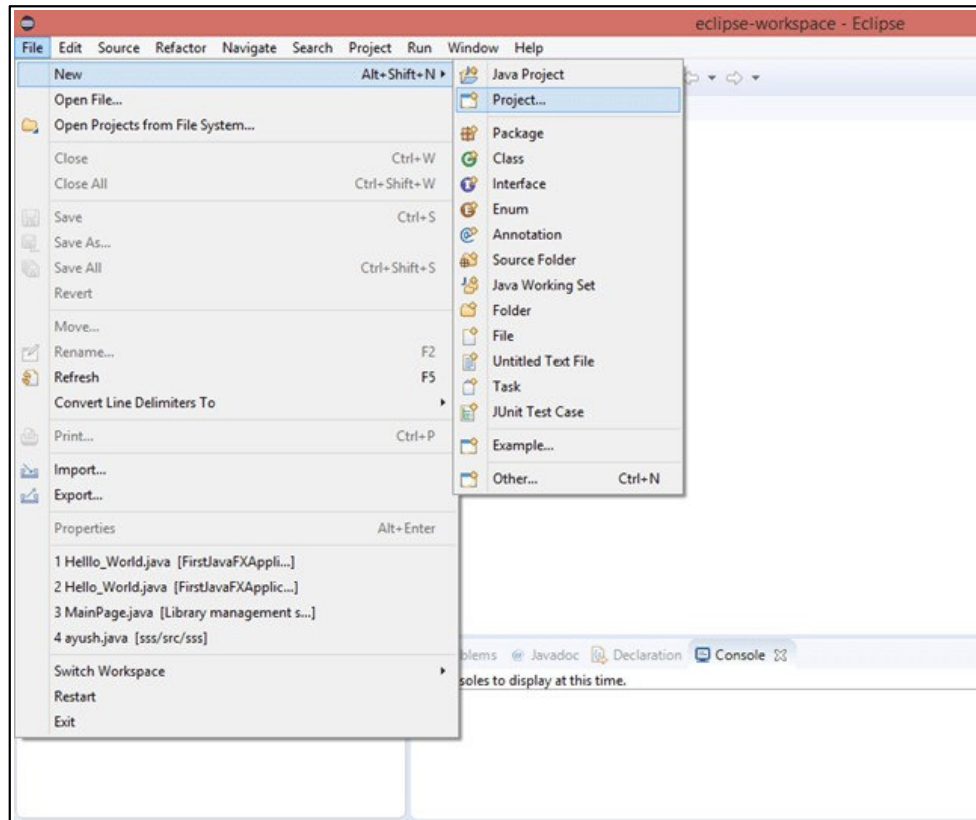
Now, a software agreement has been shown. Just select the radio box which says **I accept the terms and conditions** and click **finish**.



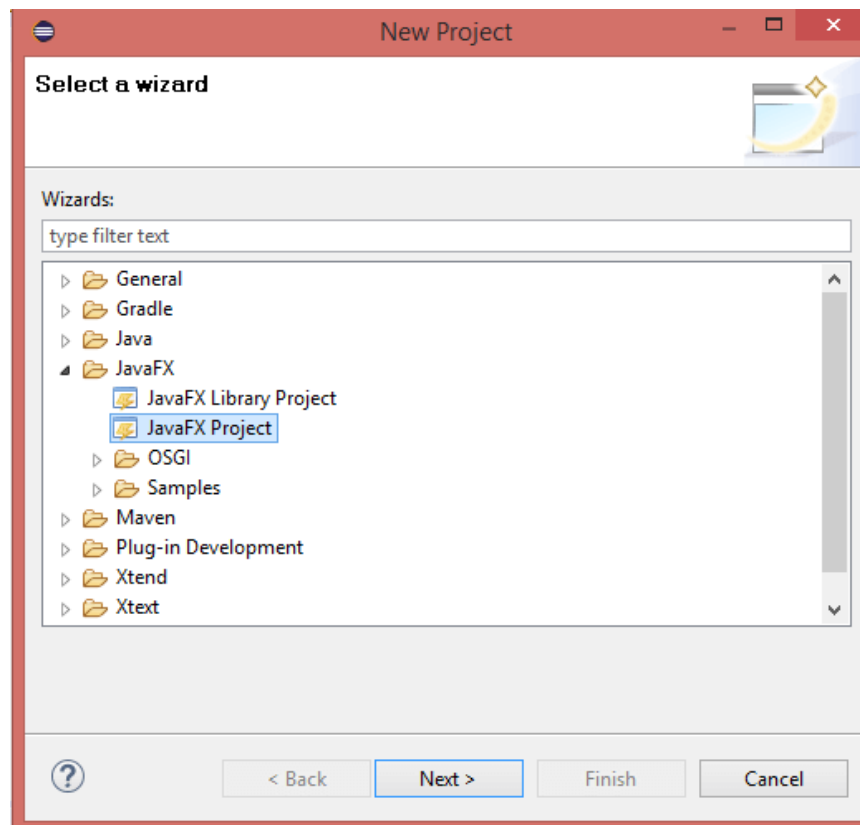
Now, the software is being installing to the eclipse. Once the process gets over, it shows a pop-up window which asks to restart the eclipse. Just click **Restart Now** and wait while restart process gets done.



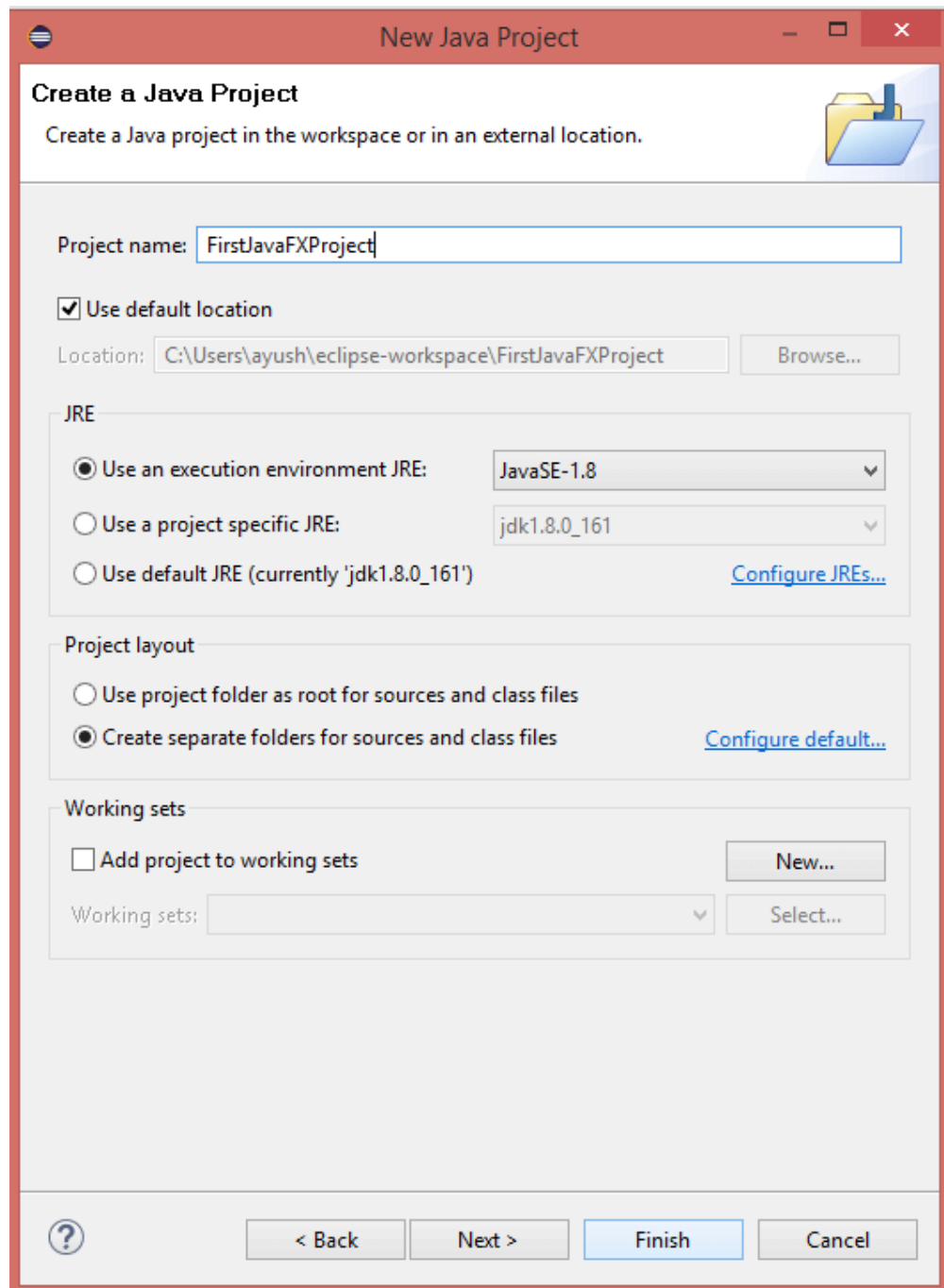
Now, we have installed the new software which enables us to create the new JavaFX Application directly. We just need to click on **file** and choose **new** → **project** from the drop-down options.



Now, we can see that there is a option of JavaFX project. We can now create JavaFX application directly. Choose **JavaFX Project** and click **Next**.



Enter the JavaFX project details and click **Finish**



The screenshot shows the 'New Java Project' dialog box in the Eclipse IDE. The dialog has a title bar with the text 'New Java Project' and standard window controls. Below the title bar, the main heading is 'Create a Java Project', followed by the instruction 'Create a Java project in the workspace or in an external location.' and a folder icon. The dialog is divided into several sections: 'Project name' with a text field containing 'FirstJavaFXProject'; 'Use default location' which is checked, with a 'Location' text field showing 'C:\Users\ayush\eclipse-workspace\FirstJavaFXProject' and a 'Browse...' button; 'JRE' section with three radio buttons: 'Use an execution environment JRE:' (selected) with a dropdown showing 'JavaSE-1.8', 'Use a project specific JRE:' with a dropdown showing 'jdk1.8.0_161', and 'Use default JRE (currently 'jdk1.8.0_161')' with a link to 'Configure JREs...'; 'Project layout' section with two radio buttons: 'Use project folder as root for sources and class files' and 'Create separate folders for sources and class files' (selected) with a link to 'Configure default...'; and 'Working sets' section with a checkbox 'Add project to working sets' (unchecked), a 'New...' button, a 'Working sets:' dropdown, and a 'Select...' button. At the bottom, there is a help icon, and navigation buttons: '< Back', 'Next >', 'Finish' (highlighted in blue), and 'Cancel'.

Create a Java Project
Create a Java project in the workspace or in an external location.

Project name:

☒ Use default location
Location:

JRE

☒ Use an execution environment JRE:

☐ Use a project specific JRE:

☐ Use default JRE (currently 'jdk1.8.0_161') [Configure JREs...](#)

Project layout

☐ Use project folder as root for sources and class files

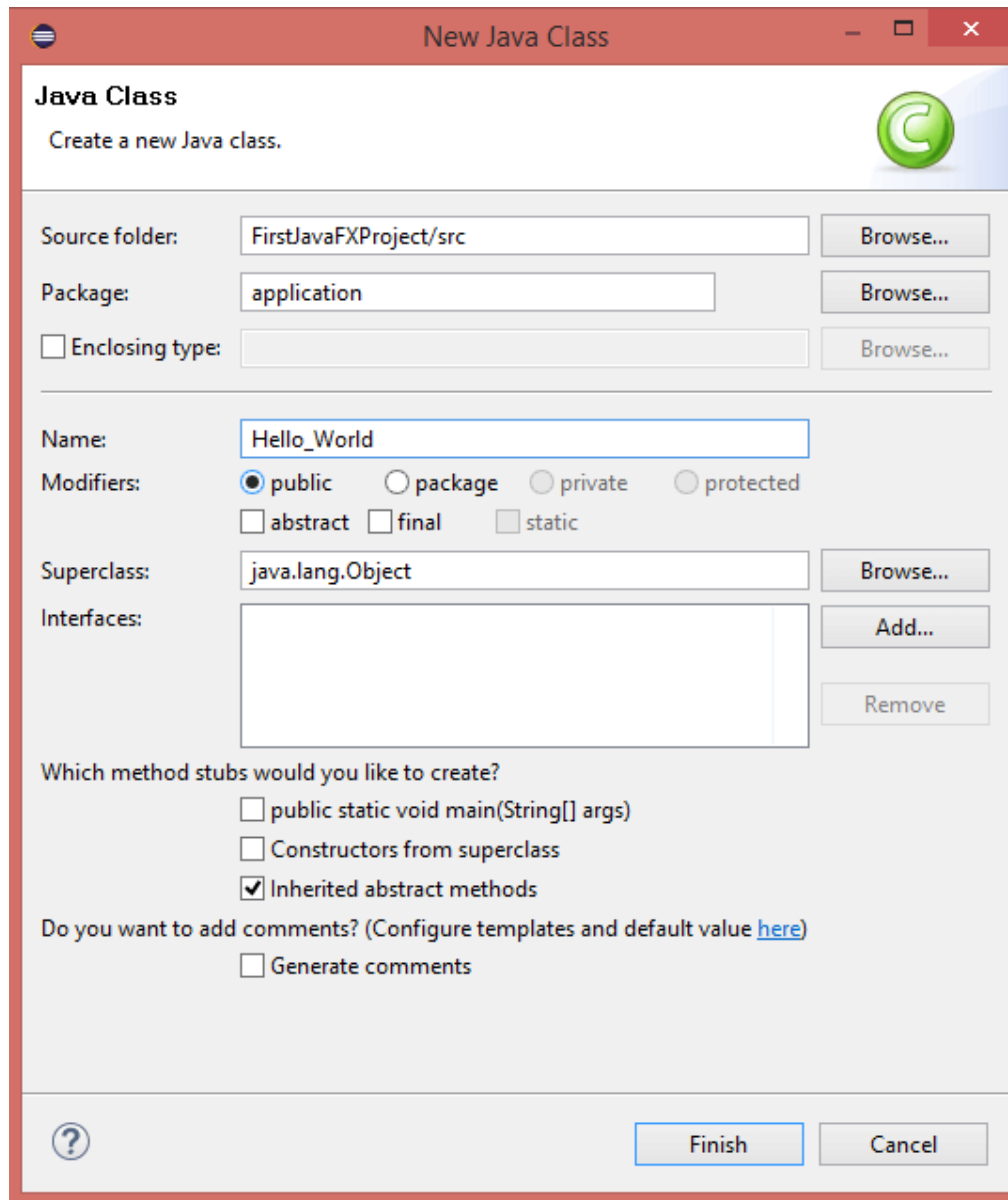
☒ Create separate folders for sources and class files [Configure default...](#)

Working sets

☐ Add project to working sets

Working sets:

Now, create your first JavaFX class. Just enter the class name in the pop-up window shown below. Click **Finish**.



New Java Class

Create a new Java class.

Source folder:

Package:

☐ Enclosing type:

Name:

Modifiers: ☒ public ☐ package ☐ private ☐ protected
☐ abstract ☐ final ☐ static

Superclass:

Interfaces:

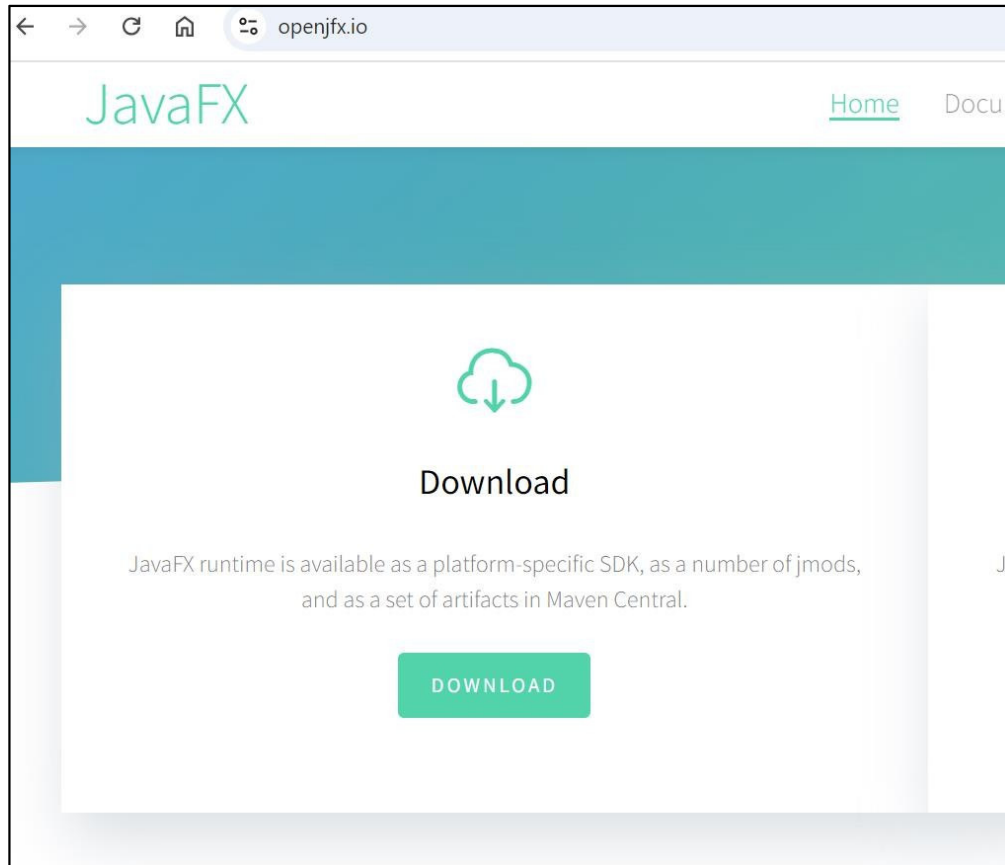
Which method stubs would you like to create?

☐ public static void main(String[] args)
☐ Constructors from superclass
☒ Inherited abstract methods

Do you want to add comments? (Configure templates and default value [here](#))
☐ Generate comments

If you get any errors to the project folder and class, do the following

Download Java FX jar files from official Java FX website <https://openjfx.io/>

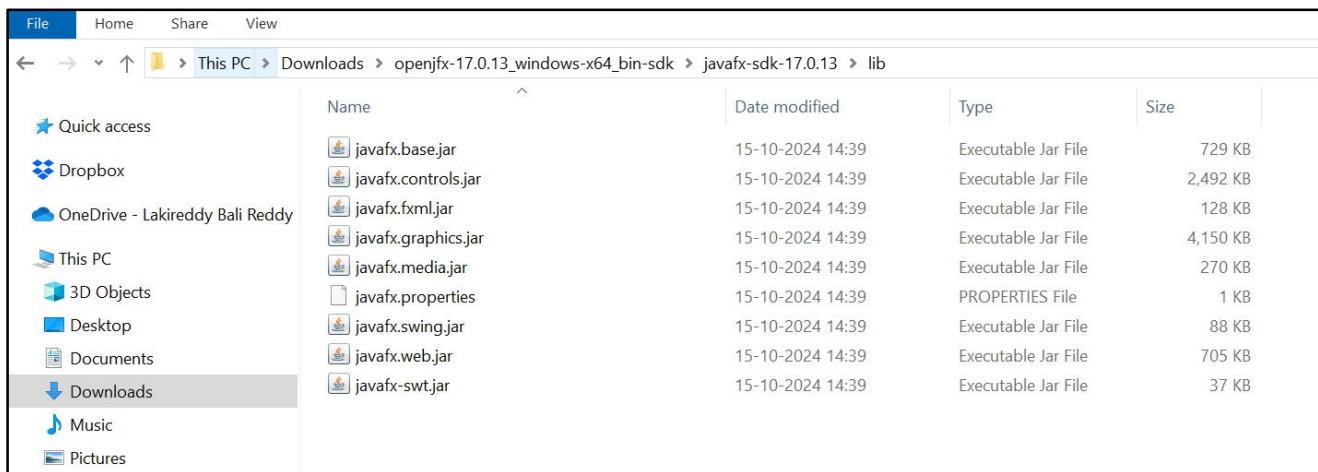


Download Windows SDK

A screenshot of the gluhq.com/products/javafx/ website. The browser address bar shows 'gluhq.com/products/javafx/'. The page has a blue header with the 'GLUON' logo on the left and navigation links: 'Products', 'Developers', 'Pricing', 'Services', 'Insights', and 'Contact'. Below the header is a table with columns for OS, Version, Architecture, and Download links. The table lists various JavaFX SDKs and jmods for Windows, Linux, and macOS, including versions 21.0.5, 17.0.13, and 23.0.1. Each row has a 'Download' button and a '[SHA256]' link.

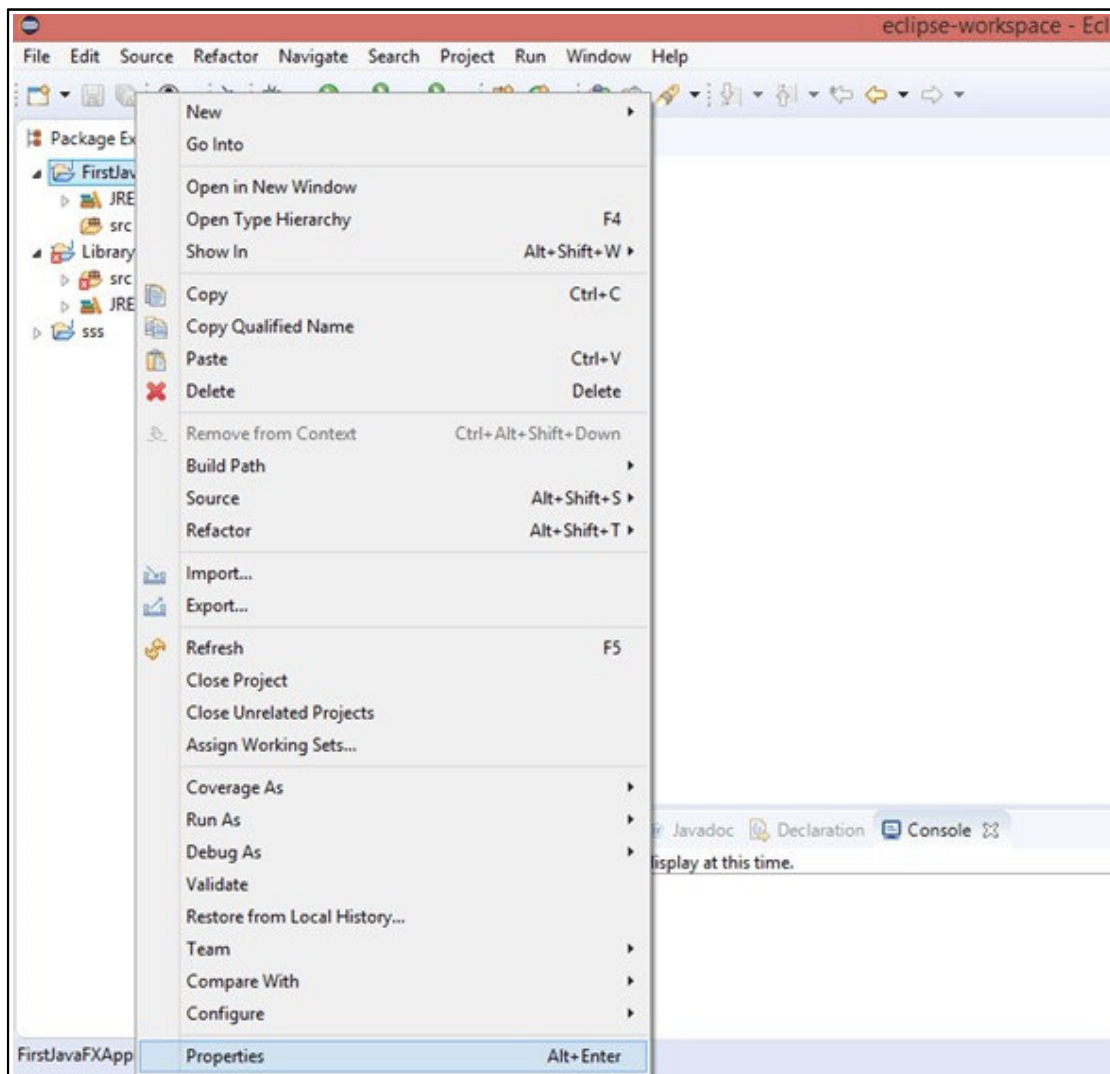
OS	Version	Architecture	Download	[SHA256]
Windows	21.0.5	x64	jmods	Download [SHA256]
Javadoc	21.0.5		Javadoc	Download [SHA256]
Linux	17.0.13	x64	SDK	Download [SHA256]
Linux	17.0.13	x64	jmods	Download [SHA256]
macOS	17.0.13	aarch64	SDK	Download [SHA256]
macOS	17.0.13	aarch64	jmods	Download [SHA256]
macOS	17.0.13	x64	SDK	Download [SHA256]
macOS	17.0.13	x64	jmods	Download [SHA256]
Windows	17.0.13	x64	SDK	Download [SHA256]
Windows	17.0.13	x64	jmods	Download [SHA256]
Javadoc	17.0.13		Javadoc	Download [SHA256]
Linux	23.0.1	aarch64	SDK	Download [SHA256]

You will get the set of jar files related to Java FX

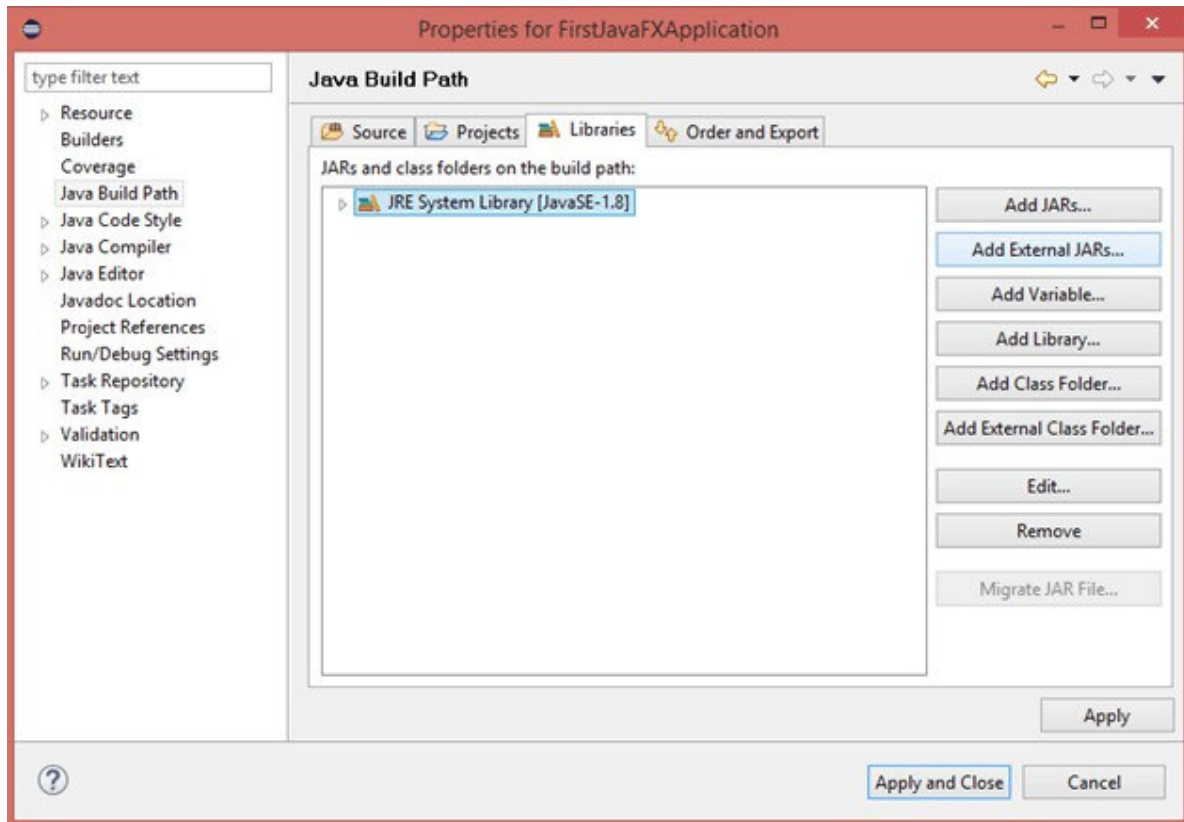


Name	Date modified	Type	Size
javafx.base.jar	15-10-2024 14:39	Executable Jar File	729 KB
javafx.controls.jar	15-10-2024 14:39	Executable Jar File	2,492 KB
javafx.fxml.jar	15-10-2024 14:39	Executable Jar File	128 KB
javafx.graphics.jar	15-10-2024 14:39	Executable Jar File	4,150 KB
javafx.media.jar	15-10-2024 14:39	Executable Jar File	270 KB
javafx.properties	15-10-2024 14:39	PROPERTIES File	1 KB
javafx.swing.jar	15-10-2024 14:39	Executable Jar File	88 KB
javafx.web.jar	15-10-2024 14:39	Executable Jar File	705 KB
javafx-swt.jar	15-10-2024 14:39	Executable Jar File	37 KB

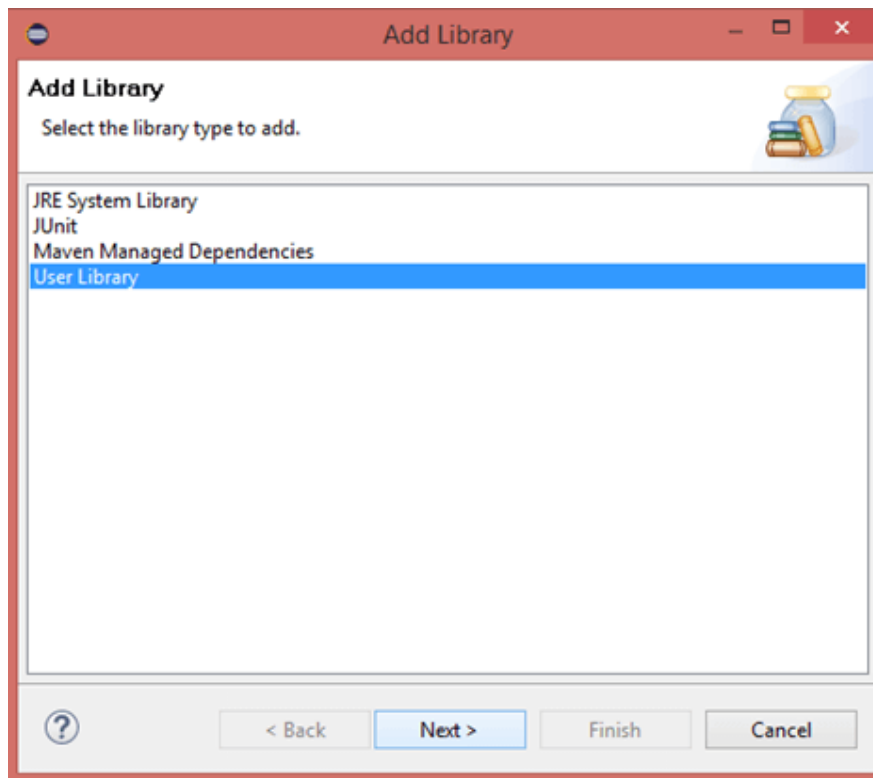
We need to export the JavaFX jar files to the project in order to run the JavaFX application. Just Right click on the project and select properties from the options.

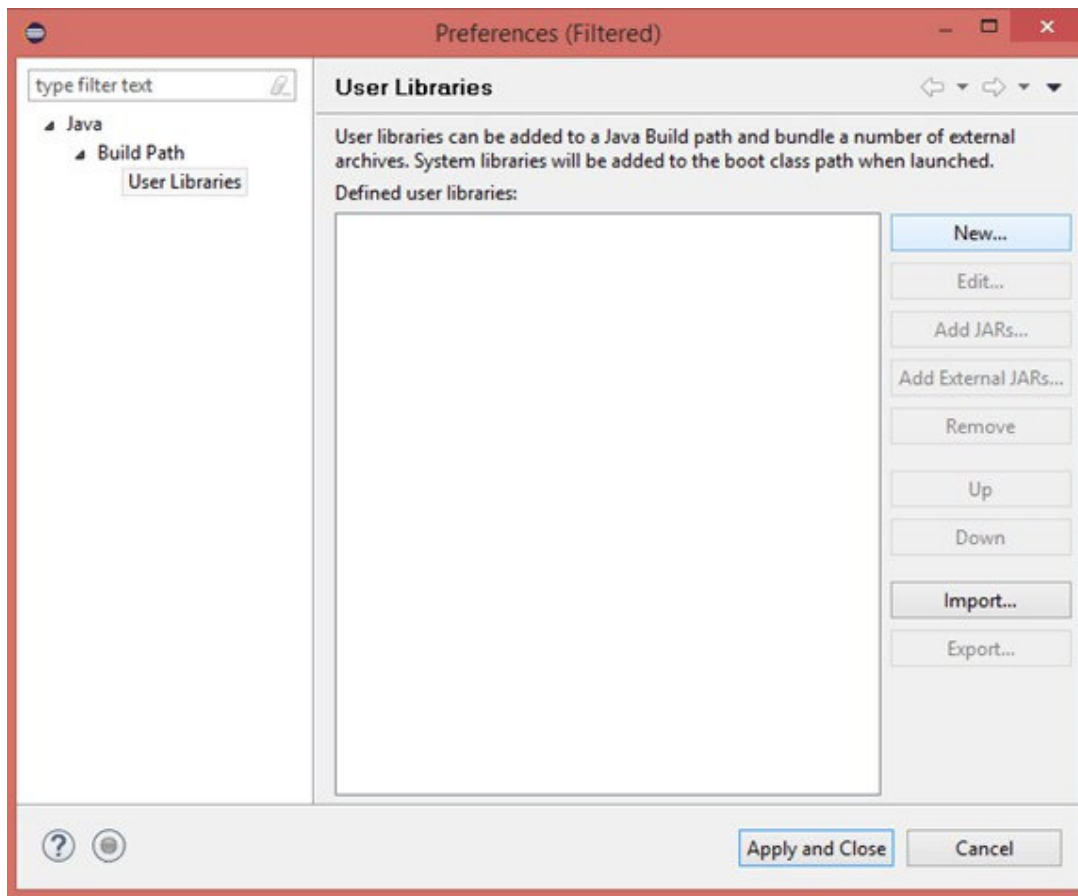


Go to **Java Build Path** → **Libraries**. Here, the JRE System Library is shown which is responsible to execute our java project. We need Add a Library here, choose **Add Library** from the options in the list given on the right side of the screen.

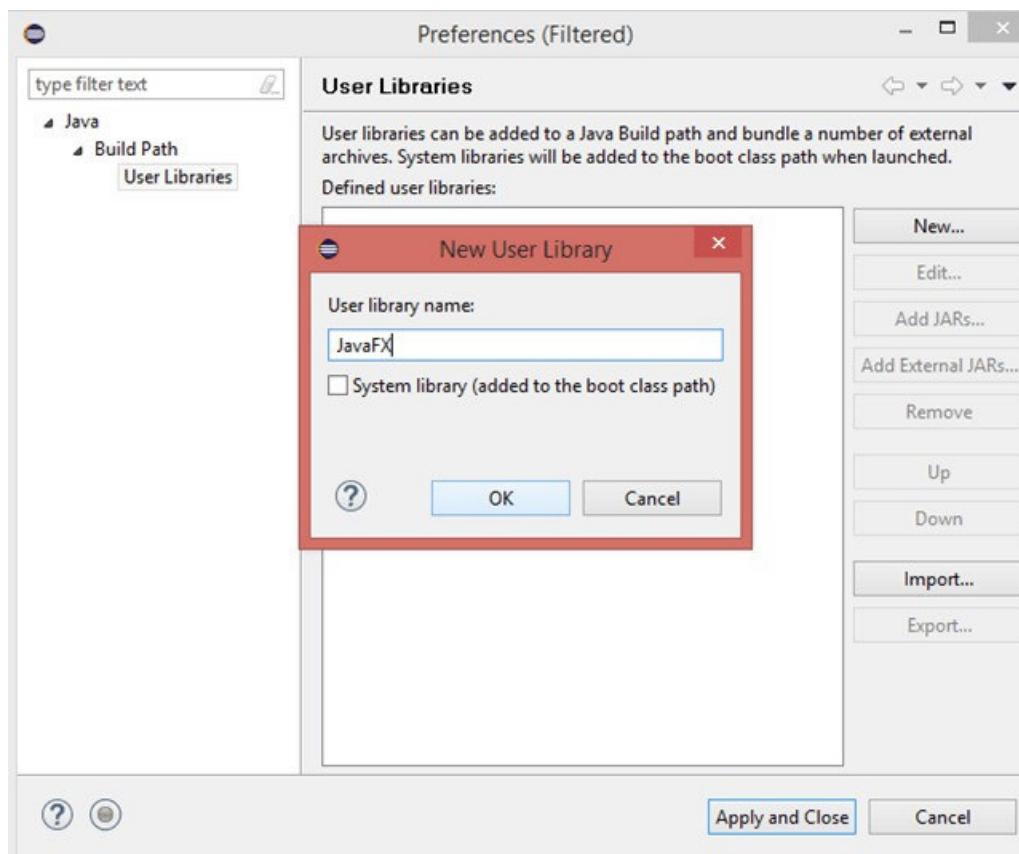


Choose **User Library** from the list and click **Next** to continue.

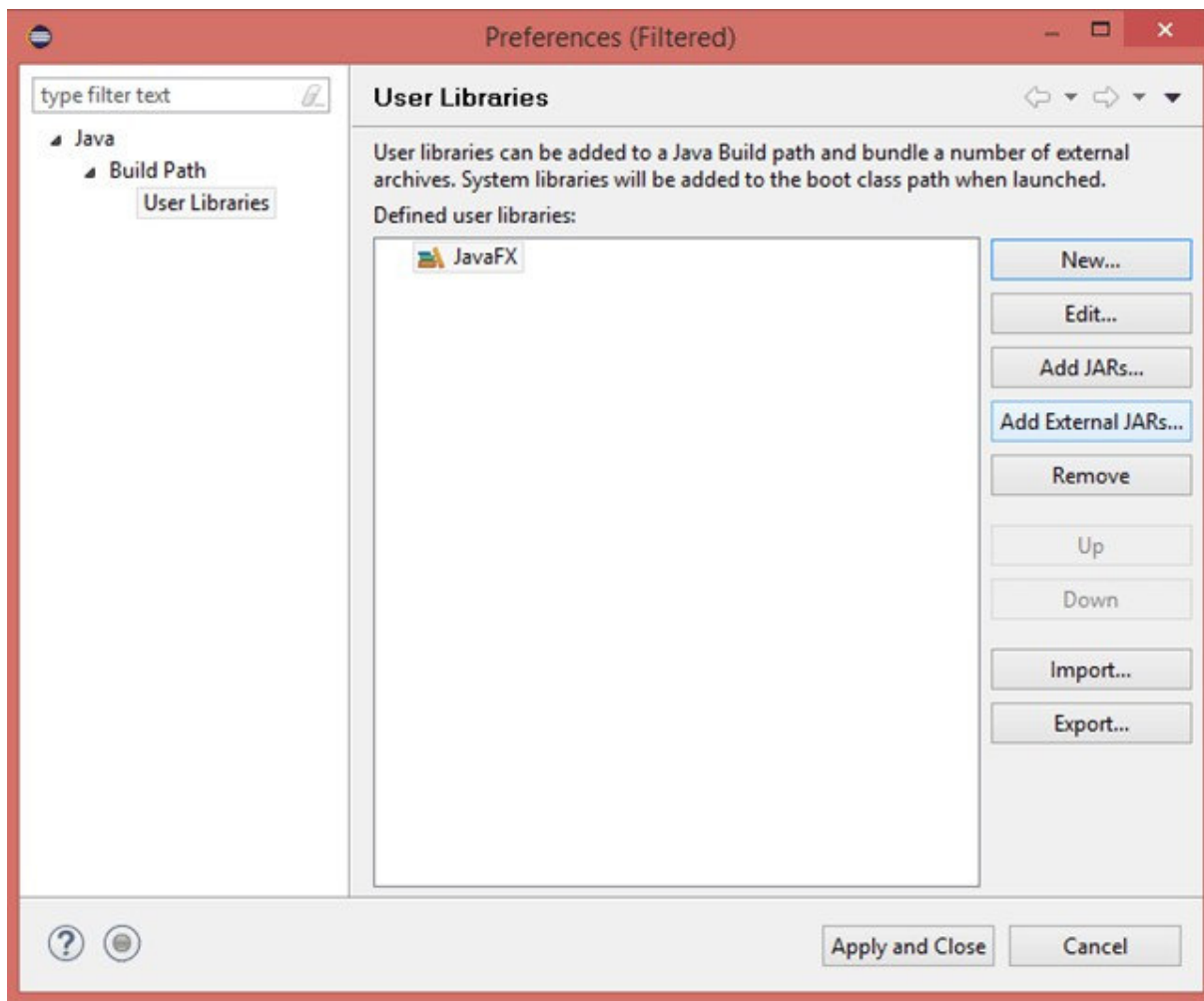




Now, We are in the user library section. We need to create a new User Library hence choose **New** from the list given on the right side of the system.

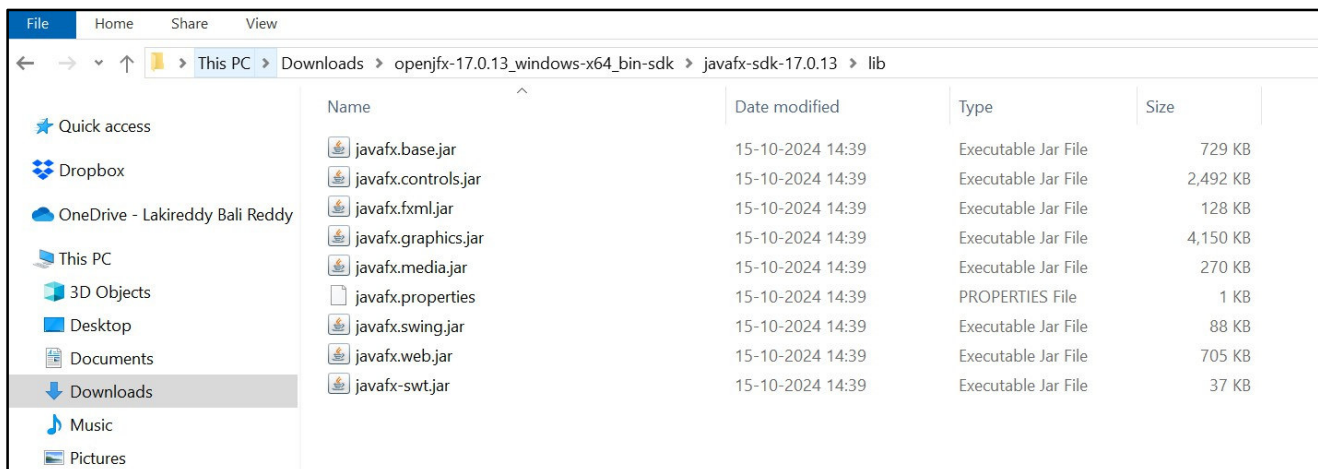


Just enter **JavaFX** in the user library name and press **OK**.



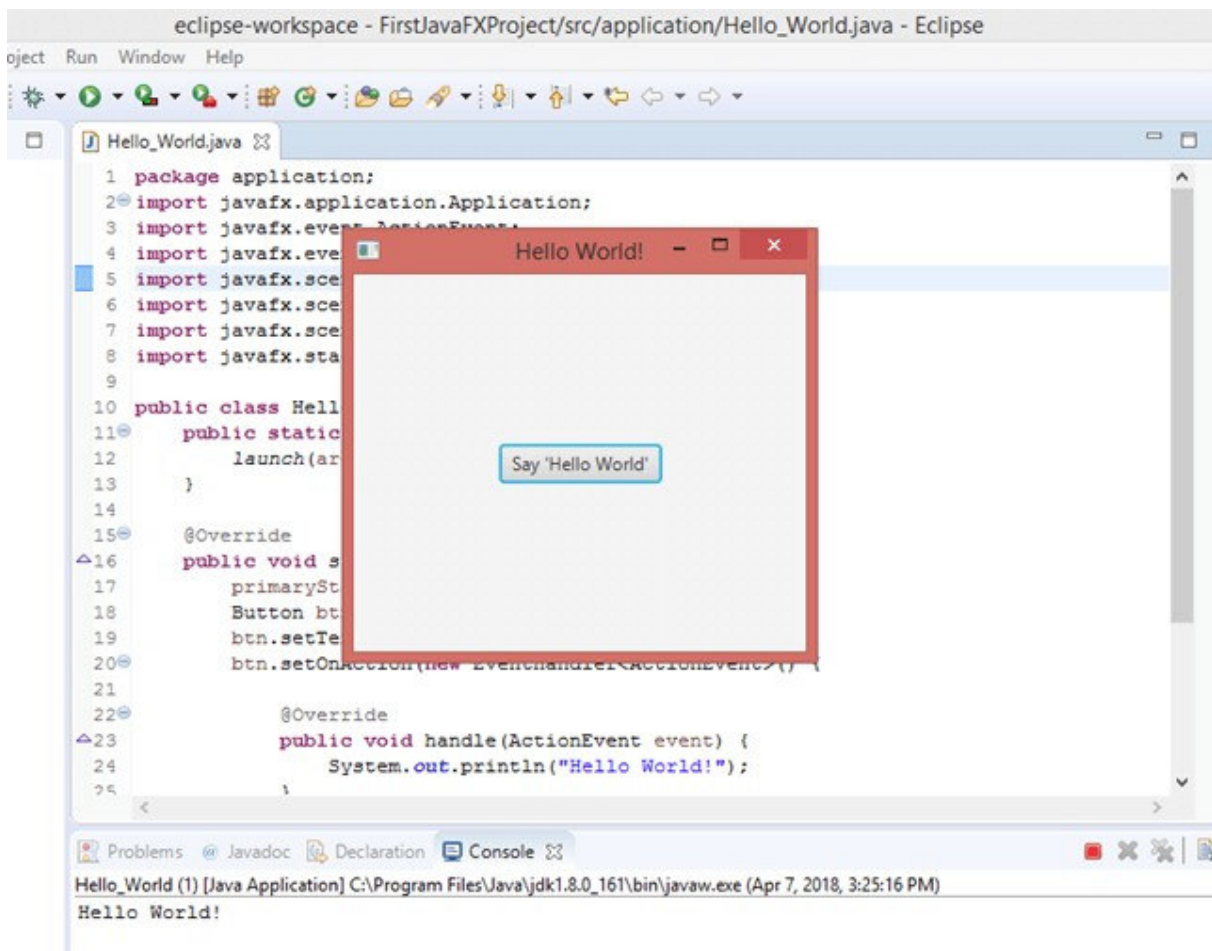
Now, a user library named as **JavaFX** has been created, we need to add some external JARs file here to make our project working.

Click on **Add External JARs** from the list given on the right side of the system and choose appropriate JAR files from the system. Add all the below jar files



Now, we are all ready to execute our first class file of the project. We have added the number of JAR files (shown in the image below) to our project. Click on **Apply and Close** to apply the settings to the project.

Right some Java FX code and execute it



We have just executed our first JavaFX application by exporting JAR files to the project.

Topic-3: JavaFX Application Structure

JavaFX application is divided hierarchically into three main components known as Stage, Scene and nodes. We need to import **javafx.application.Application** class in every JavaFX application. This provides the following life cycle methods for JavaFX application.

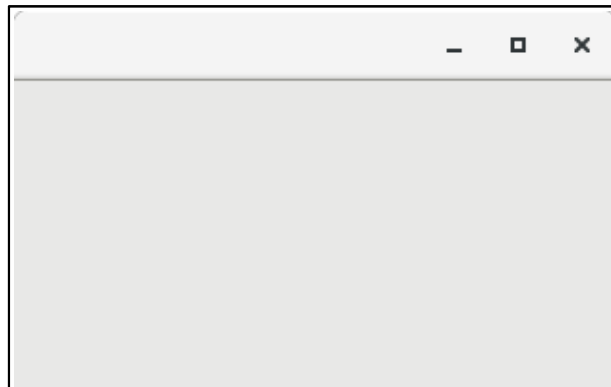
- public void init()
- public abstract void start(Stage primaryStage)
- public void stop()

in order to create a basic JavaFX application, we need to:

1. Import **javafx.application.Application** into our code.
2. Inherit **Application** into our class.
3. Override **start()** method of Application class.

Stage

Stage in a JavaFX application is similar to the Frame in a Swing Application. It acts like a container for all the JavaFX objects. Primary Stage is created internally by the platform. Other stages can further be created by the application. The object of primary stage is passed to start method. We need to call show method on the primary stage object in order to show our primary stage. Initially, the primary Stage looks like following.



However, we can add various objects to this primary stage. The objects can only be added in a hierarchical way i.e. first, scene graph will be added to this primaryStage and then that scene graph may contain the nodes. A node may be any object of the user's interface like text area, buttons, shapes, media, etc.

Scene

Scene actually holds all the physical contents (nodes) of a JavaFX application. `Javafx.scene.Scene` class provides all the methods to deal with a scene object. Creating scene is necessary in order to visualize the contents on the stage.

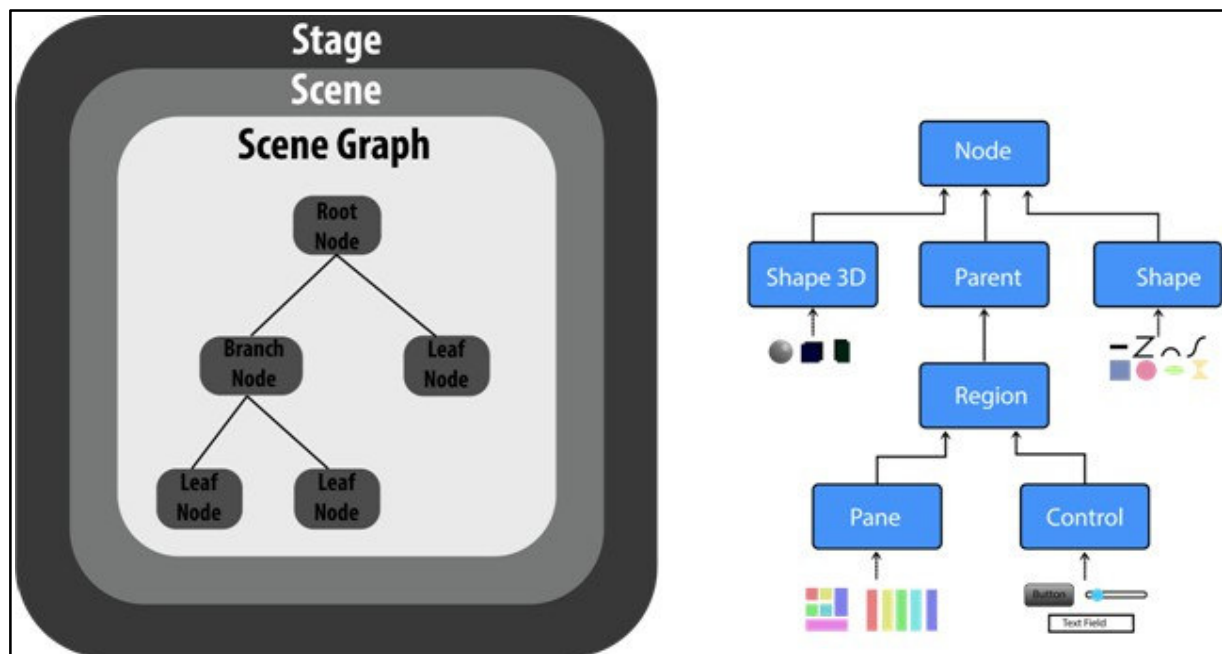
At one instance, the scene object can only be added to one stage. In order to implement Scene in our JavaFX application, we must import `javafx.scene` package in our code. The Scene can be created by creating the Scene class object and passing the layout object into the Scene class constructor.

Scene Graph

Scene Graph exists at the lowest level of the hierarchy. It can be seen as the collection of various nodes. A node is the element which is visualized on the stage. It can be any button, text box, layout, image, radio button, check box, etc.

The nodes are implemented in a tree kind of structure. There is always one root in the scene graph. This will act as a parent node for all the other nodes present in the scene graph. However, this node may be any of the layouts available in the JavaFX system.

The leaf nodes exist at the lowest level in the tree hierarchy. Each of the node present in the scene graphs represents classes of javafx.scene package therefore we need to import the package into our application in order to create a full featured javafx application.



Topic-4: Creating Our first JavaFX Application and Event Handling

Here, we are creating a simple JavaFX application which prints hello world on the console on clicking the button shown on the stage.

Step 1: Extend `javafx.application.Application` and override `start()`

As we have studied earlier that `start()` method is the starting point of constructing a JavaFX application therefore we need to first override start method of `javafx.application.Application` class. Object of the class `javafx.stage.Stage` is passed into the `start()` method therefore import this class and pass its object into start method. `JavaFX.application.Application` needs to be imported in order to override start method.

The code will look like following.

```
package application;
import javafx.application.Application;
import javafx.stage.Stage;

public class Hello_World extends Application
{
    @Override
    public void start(Stage primaryStage) throws Exception
    {
        // TODO Auto-generated method stub
    }
}
```

Step 2: Create a Button

A button can be created by instantiating the **`javafx.scene.control.Button`** class. For this, we have to import this class into our code. Pass the button label text in Button class constructor.

The code will look like following.

```
package application;
import javafx.application.Application;
import javafx.scene.control.Button;
import javafx.stage.Stage;
public class Hello_World extends Application
{
    @Override
    public void start(Stage primaryStage) throws Exception
    {
        // TODO Auto-generated method stub
        Button btn1=new Button("Say, Hello World");
    }
}
```

Step 3: Create a layout and add button to it

JavaFX provides the number of layouts. We need to implement one of them in order to visualize the widgets properly. It exists at the top level of the scene graph and can be seen as a root node. All the other nodes (buttons, texts, etc.) need to be added to this layout.

In this application, we have implemented StackPane layout. It can be implemented by instantiating `javafx.scene.layout.StackPane` class.

The code will now look like following.

```
package application;
import javafx.application.Application;
import javafx.scene.control.Button;
import javafx.stage.Stage;
import javafx.scene.layout.StackPane;
public class Hello_World extends Application
{

    @Override
    public void start(Stage primaryStage) throws Exception
    {
        // TODO Auto-generated method stub
        Button btn1=new Button("Say, Hello World");
        StackPane root=new StackPane();
        root.getChildren().add(btn1);

    }
}
```

Step 4: Create a Scene

The layout needs to be added to a scene. Scene remains at the higher level in the hierarchy of application structure. It can be created by instantiating `javafx.scene.Scene` class. We need to pass the layout object to the scene class constructor. Our application code will now look like following.

```
package application;
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.stage.Stage;
import javafx.scene.layout.StackPane;
public class Hello_World extends Application
{

    @Override
    public void start(Stage primaryStage) throws Exception
    {
        // TODO Auto-generated method stub
        Button btn1=new Button("Say, Hello World");
        StackPane root=new StackPane();
        root.getChildren().add(btn1);
        Scene scene=new Scene(root);

    }
}
```

we can also pass the width and height of the required stage for the scene in the Scene class constructor.

Step 5: Prepare the Stage

javafx.stage.Stage class provides some important methods which are required to be called to set some attributes for the stage. We can set the title of the stage. We also need to call `show()` method without which, the stage won't be shown. Lets look at the code which describes how can be prepare the stage for the application.

```
package application;
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.stage.Stage;
import javafx.scene.layout.StackPane;
public class Hello_World extends Application
{

    @Override
    public void start(Stage primaryStage) throws Exception
    {
        // TODO Auto-generated method stub
        Button btn1=new Button("Say, Hello World");
        StackPane root=new StackPane();
        root.getChildren().add(btn1); Scene
        scene=new Scene(root);
        primaryStage.setScene(scene);
        primaryStage.setTitle("First JavaFX Application");
        primaryStage.show();
    }
}
```

Step 6: Create an event for the button

As our application prints hello world for an event on the button. We need to create an event for the button. For this purpose, call **setOnAction()** on the button and define a anonymous class Event Handler as a parameter to the method.

Inside this anonymous class, define a method handle() which contains the code for how the event is handled. In our case, it is printing hello world on the console.

```
package application;
import javafx.application.Application;
import javafx.event.ActionEvent; import
javafx.event.EventHandler; import
javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.stage.Stage;
import javafx.scene.layout.StackPane;
public class Hello_World extends Application
{

    @Override
    public void start(Stage primaryStage) throws Exception
    {
        // TODO Auto-generated method stub
        Button btn1=new Button("Say, Hello World"); btn1.setOnAction(new
        EventHandler<ActionEvent>()
        {

            @Override
            public void handle(ActionEvent arg0) {
                // TODO Auto-generated method stub System.out.println("hello
                world");
            }
        });
        StackPane root=new StackPane();
        root.getChildren().add(btn1);
        Scene scene=new Scene(root,600,400); primaryStage.setScene(scene);
        primaryStage.setTitle("First JavaFX Application");
        primaryStage.show();
    }
}
```

Step 7: Create the main method

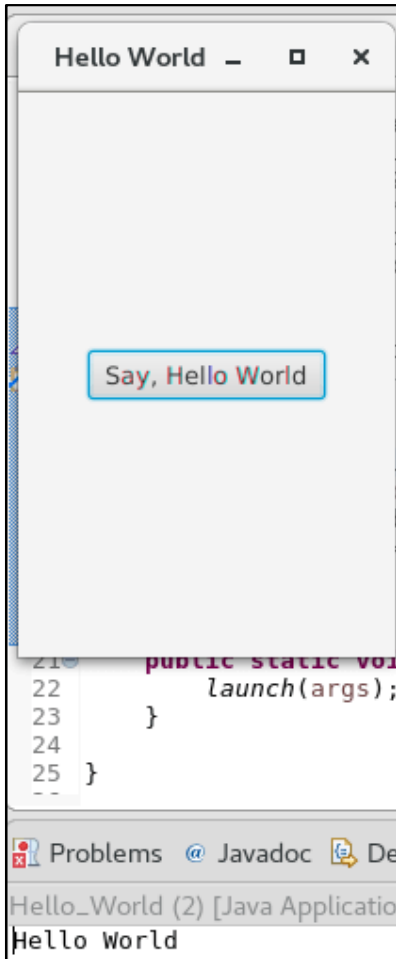
Till now, we have configured all the necessary things which are required to develop a basic JavaFX application but this application is still incomplete. We have not created main method yet. Hence, at the last, we need to create a main method in which we will launch the application i.e. will call `launch()` method and pass the command line arguments (args) to it. The code will now look like following.

```
package application;
import javafx.application.Application;
import javafx.event.ActionEvent; import
javafx.event.EventHandler; import
javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.stage.Stage;
import javafx.scene.layout.StackPane;
public class Hello_World extends Application
{

    @Override
    public void start(Stage primaryStage) throws Exception
    {
        // TODO Auto-generated method stub
        Button btn1=new Button("Say, Hello World"); btn1.setOnAction(new
        EventHandler<ActionEvent>()
        {

            @Override
            public void handle(ActionEvent arg0) {
                // TODO Auto-generated method stub System.out.println("hello
                world");
            }
        });
        StackPane root=new StackPane();
        root.getChildren().add(btn1);
        Scene scene=new Scene(root,600,400);
        primaryStage.setTitle("First JavaFX Application");
        primaryStage.setScene(scene);
        primaryStage.show();
    }
    public static void main (String[] args)
    {
        launch(args);
    }
}
```

The Application will produce the following output on the screen.



Topic-4: Event Handling in JavaFX

- In JavaFX, event handling is the mechanism by which a program reacts to user interactions, like button clicks, keyboard presses, or mouse movements.
 - JavaFX uses an event-driven model, meaning that specific event handlers are registered to listen for and respond to events.
1. **Event Source:** The source of the event (e.g., a `Button`, `TextField`, etc.).
 2. **Event Object:** Contains information about the event, like type, timestamp, and source.
 3. **Event Handler:** The code that processes the event, typically implemented using a lambda function or an anonymous inner class.

Steps to Handle Events

1. Create an *event source* (like a button).
2. Attach an *event handler* to the source.
3. The handler will automatically execute when the specified event occurs.

Example: Button Click Event

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.control.Label;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;

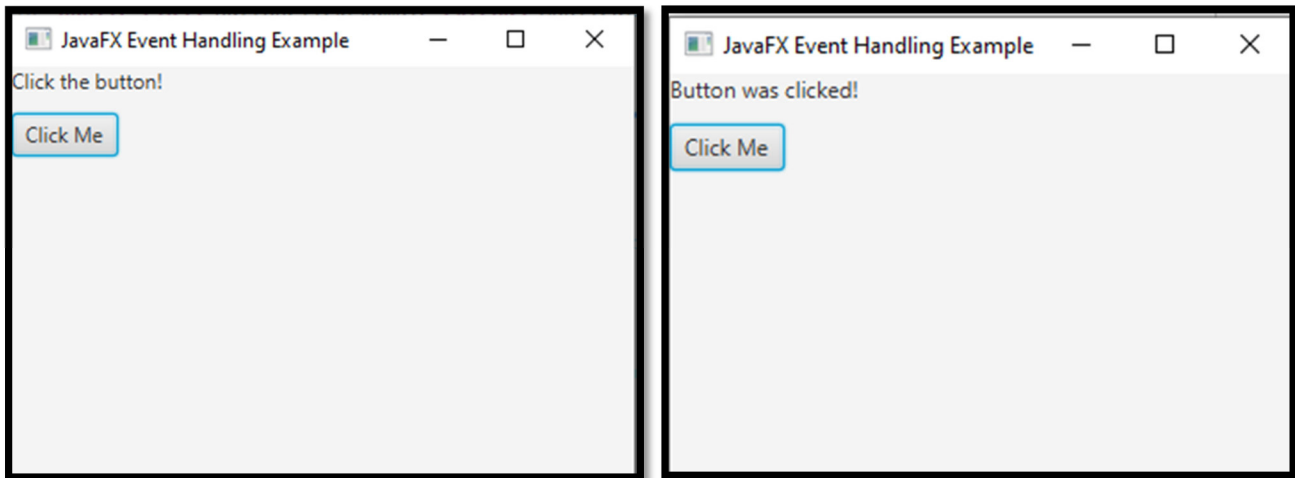
public class ButtonClickExample extends Application {
    @Override
    public void start(Stage primaryStage) {
        // Create a Label
        Label label = new Label("Click the button!");
        // Create a Button
        Button button = new Button("Click Me");

        // Set up the event handler for the button
        button.setOnAction(event -> {
            label.setText("Button was clicked!");
        });

        // Create a layout and add the button and label
        VBox vbox = new VBox(10, label, button);
        Scene scene = new Scene(vbox, 300, 200);

        // Set up the stage
        primaryStage.setTitle("JavaFX Event Handling Example");
        primaryStage.setScene(scene);
        primaryStage.show();
    }
    public static void main(String[] args) {
        launch(args);
    }
}
```

Output:



Topic-5: Laying out nodes in scene graph in javafx

- In JavaFX, layout management is handled through **layout panes** that position nodes in a scene graph.
- The scene graph is a hierarchical structure where each node is either a parent (containing child nodes) or a leaf (having no children).
- Layout panes automatically arrange and align nodes based on the pane's rules, making it easy to create visually organized and responsive UIs.

Layout Panes in JavaFX

1. **Pane**: The base class for all layout panes. Provides no automatic layout behavior.
2. **HBox**: Lays out its children horizontally in a single row.
3. **VBox**: Lays out its children vertically in a single column.
4. **StackPane**: Stacks children on top of each other in the center.
5. **BorderPane**: Divides the layout into five regions: top, bottom, left, right, and center.
6. **GridPane**: Organizes nodes in a grid, defined by rows and columns.
7. **AnchorPane**: Allows anchoring nodes to the edges of the pane, which is helpful for dynamic resizing.

Example: Laying Out Nodes Using Different Layout Panes

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.control.Label;
import javafx.scene.layout.*;
import javafx.stage.Stage;

public class LayoutExample extends Application {
    @Override
    public void start(Stage primaryStage) {
```

```

        // Create a Label in the top region
        Label topLabel = new Label("Top Region");
        BorderPane.setAlignment(topLabel,
javafx.geometry.Pos.CENTER);

        // HBox in the center with buttons arranged horizontally
        HBox hbox = new HBox(10);
        hbox.getChildren().addAll(new Button("Button 1"), new
Button("Button 2"), new Button("Button 3"));

        // VBox in the left region with labels arranged vertically
        VBox vbox = new VBox(10);
        vbox.getChildren().addAll(new Label("Left 1"), new
Label("Left 2"), new Label("Left 3"));

        // GridPane in the bottom region with labels in a grid
        GridPane gridPane = new GridPane();
        gridPane.setHgap(10);
        gridPane.setVgap(10);
        gridPane.add(new Label("Row 0, Col 0"), 0, 0);
        gridPane.add(new Label("Row 0, Col 1"), 1, 0);
        gridPane.add(new Label("Row 1, Col 0"), 0, 1);
        gridPane.add(new Label("Row 1, Col 1"), 1, 1);

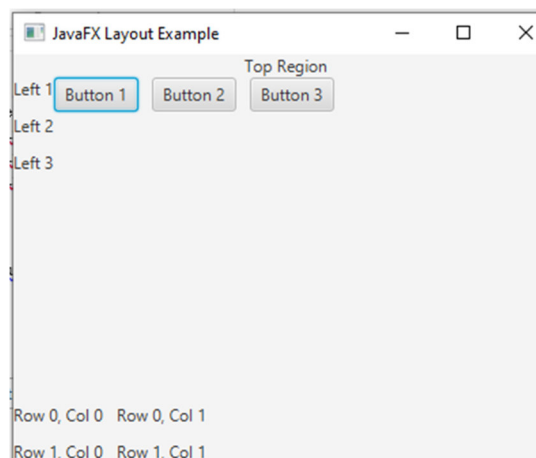
        // Create a BorderPane as the root layout
        BorderPane borderPane = new BorderPane();
        borderPane.setTop(topLabel);
        borderPane.setCenter(hbox);
        borderPane.setLeft(vbox);
        borderPane.setBottom(gridPane);

        // Create a scene and set it to the stage
        Scene scene = new Scene(borderPane, 400, 300);
        primaryStage.setTitle("JavaFX Layout Example");
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}

```

Output:



Topic-6: Mouse Events in javafx

- Mouse events in JavaFX allow developers to respond to user interactions with the mouse, such as clicks, movements, drags, scrolls, and more.
- These events are handled by attaching event listeners to nodes, which then respond to specific mouse actions.

Common Mouse Events in JavaFX

JavaFX provides several types of mouse events through the `MouseEvent` class. The most common ones are:

1. **MOUSE_CLICKED**: Triggered when the mouse is clicked (pressed and released) on a node.
2. **MOUSE_PRESSED**: Triggered when a mouse button is pressed on a node.
3. **MOUSE_RELEASED**: Triggered when a mouse button is released on a node.
4. **MOUSE_MOVED**: Triggered when the mouse pointer moves over a node.
5. **MOUSE_DRAGGED**: Triggered when the mouse is pressed and dragged over a node.
6. **MOUSE_ENTERED**: Triggered when the mouse pointer enters a node.
7. **MOUSE_EXITED**: Triggered when the mouse pointer exits a node.

Each event contains details about the mouse action, such as the (x, y) coordinates of the mouse pointer, the button pressed, and whether modifier keys (like `Shift` or `Ctrl`) were held.

Example: Handling Mouse Events in JavaFX

Below is an example that demonstrates various mouse events. It includes a `Rectangle` node that changes color on different mouse actions and displays the mouse coordinates when moved.

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.input.MouseEvent;
import javafx.scene.paint.Color;
import javafx.scene.shape.Rectangle;
import javafx.scene.control.Label;
import javafx.scene.layout.StackPane;
import javafx.stage.Stage;

public class MouseEventsExample extends Application {
    @Override
    public void start(Stage primaryStage) {
        // Create a Rectangle
        Rectangle rectangle = new Rectangle(100, 100,
Color.LIGHTBLUE);

        // Create a Label to show mouse coordinates
        Label coordinatesLabel = new Label("Mouse Coordinates: ");

        // Handle MOUSE_ENTERED event
```

```

rectangle.setOnMouseEntered(event -> {
    rectangle.setFill(Color.LIGHTGREEN);
    coordinatesLabel.setText("Mouse Entered the Rectangle");
});

// Handle MOUSE_EXITED event
rectangle.setOnMouseExited(event -> {
    rectangle.setFill(Color.LIGHTBLUE);
    coordinatesLabel.setText("Mouse Exited the Rectangle");
});

// Handle MOUSE_CLICKED event
rectangle.setOnMouseClicked(event -> {
    rectangle.setFill(Color.ORANGE);
    coordinatesLabel.setText("Rectangle Clicked!");
});

// Handle MOUSE_PRESSED event
rectangle.setOnMousePressed(event -> {
    rectangle.setFill(Color.DARKBLUE);
    coordinatesLabel.setText("Mouse Pressed on Rectangle");
});

// Handle MOUSE_RELEASED event
rectangle.setOnMouseReleased(event -> {
    rectangle.setFill(Color.ORANGE);
    coordinatesLabel.setText("Mouse Released on Rectangle");
});

// Handle MOUSE_MOVED event
rectangle.setOnMouseMoved(event -> {
    coordinatesLabel.setText(String.format("Mouse Moved to
[%.1f, %.1f]", event.getX(), event.getY()));
});

// Handle MOUSE_DRAGGED event
rectangle.setOnMouseDragged(event -> {
    coordinatesLabel.setText(String.format("Mouse Dragged to
[%.1f, %.1f]", event.getX(), event.getY()));
});

// Set up the layout
StackPane root = new StackPane();
root.getChildren().addAll(rectangle, coordinatesLabel);

// Position the label above the rectangle
StackPane.setMargin(coordinatesLabel, new
javafx.geometry.Insets(120, 0, 0, 0));

// Create a Scene and display the stage
Scene scene = new Scene(root, 300, 250);
primaryStage.setTitle("JavaFX Mouse Events Example");
primaryStage.setScene(scene);
primaryStage.show();
}

```

```
    public static void main(String[] args) {  
        launch(args);  
    }  
}
```

Output:

